

# Laboratorio de repaso: Matrices en C++

De `main()` a funciones, procedimientos y menú

Ing. Jorge Luis García Chacón

## Laboratorio: Matrices paso a paso

**Curso:** Algoritmos

**Tiempo estimado:** 60 minutos

**Modalidad:** Individual

**Herramienta:** C++ / OnlineGDB

### Propósito

Construir una solución sencilla con una matriz inicializada desde su declaración y transformarla gradualmente desde un programa escrito completamente en `main()` hasta una solución organizada con funciones, procedimientos y un menú.

## Objetivos de aprendizaje

Al finalizar el laboratorio, el estudiante será capaz de:

- Declarar e inicializar una matriz bidimensional.
- Recorrer una matriz mediante ciclos `for` anidados.
- Diferenciar el recorrido por filas y por columnas.
- Calcular totales por fila y por columna.
- Trasladar bloques de código de `main()` hacia procedimientos y funciones.
- Utilizar parámetros para enviar una matriz a otras partes del programa.
- Integrar las operaciones mediante un menú sencillo.

## 1. Situación del laboratorio

Una pequeña **panadería** registra la cantidad de productos vendidos durante tres días. Se controlan cuatro productos:

1. Pan francés
2. Pan dulce
3. Pastel
4. Galletas

Cada **fila** representa un día y cada **columna** representa un producto.

Trabajaremos con la siguiente matriz ya inicializada:

```
int ventas[3][4] = {  
    {12, 8, 5, 10},  
    {15, 6, 7, 9},  
    {10, 9, 4, 11}  
};
```

### **i** Antes de programar

Observe que la matriz tiene **3 filas y 4 columnas**. Por ejemplo, `ventas[1][2]` corresponde a la segunda fila y tercera columna porque los índices comienzan en cero.

## 2. Paso 1 — Todo dentro de `main()`

Primero construiremos la solución sin funciones ni procedimientos. El objetivo es comprender claramente el recorrido de la matriz.

### 2.1 Mostrar la matriz

Complete y ejecute:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int ventas[3][4] = {  
        {12, 8, 5, 10},  
        {15, 6, 7, 9},  
        {10, 9, 4, 11}  
    };  
  
    cout << "MATRIZ DE VENTAS" << endl;
```

```

for (int fila = 0; fila < 3; fila++) {
    for (int columna = 0; columna < 4; columna++) {
        cout << ventas[fila][columna] << "\t";
    }
    cout << endl;
}

return 0;
}

```

**Compruebe:** ¿qué ciclo cambia las filas?, ¿qué ciclo recorre las columnas?

### 3. Paso 2 — Totalizar primero las filas

Ahora necesitamos conocer cuántos productos se vendieron **en cada día**.

La idea es sencilla: fijamos una fila y recorremos todas sus columnas.

```

for (int fila = 0; fila < 3; fila++) {
    int totalFila = 0;

    for (int columna = 0; columna < 4; columna++) {
        totalFila = totalFila + ventas[fila][columna];
    }

    cout << "Total del dia " << fila + 1 << ": " << totalFila << endl;
}

```

#### Verificación manual

Para la primera fila:

$$12 + 8 + 5 + 10 = 35$$

Antes de continuar, verifique que su programa muestre **35** para el día 1.

## 4. Paso 3 — Ahora totalizar las columnas

Queremos conocer el total vendido **de cada producto** durante los tres días.

Esta vez cambia la lógica del recorrido: fijamos una columna y recorremos todas las filas.

```
for (int columna = 0; columna < 4; columna++) {
    int totalColumna = 0;

    for (int fila = 0; fila < 3; fila++) {
        totalColumna = totalColumna + ventas[fila][columna];
    }

    cout << "Total del producto " << columna + 1
          << ": " << totalColumna << endl;
}
```

! Concepto clave para el examen

**Total por fila:** el ciclo exterior controla filas.

**Total por columna:** el ciclo exterior controla columnas.

La matriz es la misma; lo que cambia es la forma en que la recorremos.

## 5. Paso 4 — Sacar la impresión de main()

Nuestro programa funciona, pero `main()` comienza a crecer. Organicemos el código.

Crearemos un **procedimiento**, porque mostrar la matriz realiza una tarea pero no necesita devolver un valor.

```
void mostrarMatriz(int ventas[][4], int filas) {
    cout << "\nMATRIZ DE VENTAS\n";

    for (int fila = 0; fila < filas; fila++) {
        for (int columna = 0; columna < 4; columna++) {
            cout << ventas[fila][columna] << "\t";
        }
        cout << endl;
    }
}
```

Desde `main()` la llamada será:

```
mostrarMatriz(ventas, 3);
```

**Pregunta de repaso:** ¿por qué `mostrarMatriz` es un procedimiento y no una función que devuelve un número?

## 6. Paso 5 — Convertir el total de una fila en función

Ahora construiremos una función que reciba la matriz y el número de fila que queremos totalizar.

```
int totalFila(int ventas[][4], int fila) {
    int total = 0;

    for (int columna = 0; columna < 4; columna++) {
        total += ventas[fila][columna];
    }

    return total;
}
```

Podemos reutilizarla así:

```
for (int fila = 0; fila < 3; fila++) {
    cout << "Dia " << fila + 1
        << ": " << totalFila(ventas, fila) << endl;
}
```

## 7. Paso 6 — Convertir el total de una columna en función

Ahora complete la función equivalente para columnas:

```
int totalColumna(int ventas[][4], int columna) {
    int total = 0;

    // Complete el ciclo que recorre las 3 filas.

    return total;
}
```

Después utilícela desde `main()` para mostrar los cuatro totales.

#### Pista

Si queremos totalizar una columna, `columna` permanece fija mientras cambia `fila`.

## 8. Paso 7 — Crear procedimientos de reporte

Para que el programa principal quede todavía más limpio, construya:

```
void mostrarTotalesFilas(int ventas[][4]) {
    // Utilice totalFila(...)
}

void mostrarTotalesColumnas(int ventas[][4]) {
    // Utilice totalColumna(...)
}
```

**Regla:** no vuelva a escribir dentro de estos procedimientos la lógica de las sumas. Deben **reutilizar las funciones** creadas anteriormente.

## 9. Paso 8 — Integrar un menú

El programa final mostrará:

```
===== PANADERIA - CONTROL DE VENTAS =====
1. Mostrar matriz
2. Mostrar totales por dia
3. Mostrar totales por producto
4. Salir
Seleccione una opcion:
```

Utilice `do...while` y `switch`:

```
do {
    cout << "\n===== MENU =====\n";
    cout << "1. Mostrar matriz\n";
    cout << "2. Totales por dia\n";
    cout << "3. Totales por producto\n";
```

```

    cout << "4. Salir\n";
    cout << "Opcion: ";
    cin >> opcion;

    switch (opcion) {
        case 1:
            mostrarMatriz(ventas, 3);
            break;
        case 2:
            mostrarTotalesFilas(ventas);
            break;
        case 3:
            mostrarTotalesColumnas(ventas);
            break;
        case 4:
            cout << "Fin del programa." << endl;
            break;
        default:
            cout << "Opcion no valida." << endl;
    }

} while (opcion != 4);

```

## 10. Reto final — Integración

Construya el programa completo. Al finalizar, `main()` debería dedicarse principalmente a:

1. Inicializar la matriz.
2. Declarar la opción.
3. Mostrar el menú.
4. Realizar llamadas a funciones y procedimientos.

No duplique la lógica que ya fue encapsulada.

## 11. Preguntas de repaso

1. ¿Qué representa una fila en este problema?
2. ¿Qué representa una columna?
3. ¿Cuántos ciclos se necesitan para recorrer completamente una matriz bidimensional?
4. ¿Cuál es la diferencia entre totalizar una fila y totalizar una columna?

5. ¿Qué ventaja obtenemos al mover código de `main()` hacia funciones y procedimientos?
6. ¿Qué función devuelve un valor en este laboratorio?
7. ¿Qué procedimientos únicamente realizan una tarea y no devuelven un resultado?

## 12. Evidencia

Entregue una captura donde se observe:

- la matriz completa;
- los totales por día;
- los totales por producto;
- el menú funcionando.

## 13. Cierre

**i** Lo importante de hoy

Una matriz no es complicada cuando se comprende su recorrido. Primero aprendimos a **verla**, luego a **recorrerla**, después a **procesarla** y finalmente a **organizar el programa** mediante funciones, procedimientos y un menú.